TECHNION · FINAL PROJECT EVALUATION

MCP Security Scanner
Final validation report

Bar David & Noam Ben Lulu · B.Sc. Electrical and Computer Engineering

Evaluation: 2026-10-16 · Submission: 21 October 2026

92.9%
Scoped precision

43.3%
Recall

17 / 30
Missed positive labels

1 / 20
False alarms

On the reserved final set, the scanner detected 13 of 30 positive rule labels and raised 1 false alarm among 20 negative labels: 92.9% precision and 43.3% recall. These results support a working heuristic prototype with substantial remaining recall limitations; they do not establish production readiness.

Data preparation

Converted 930 records: 218 prompt-injection-bench, 180 Vault v2, 498 ATR skill records, and 34 explicitly MCP-tagged Bordair records. Admitted 98 public-text records; 832 remain quarantined and unscored. Added 24 synthetic fixtures from six scenario families, with inline/helper variants and paired vulnerable/safe behavior. The three campaign rounds contain 122 labeled fixtures, reusing the 22 adapted and 24 synthetic pilot cases. They are not 122 independent incidents. Text was preserved verbatim in standard MCP tool docstrings, changing its original context. Dataset labels were never

copied blindly into scanner ground truth. MCP-FBAs preference pairs and unrelated Bordair modalities were excluded because they do not supply static-source vulnerability labels.

Completed schedule

Completed order: dataset conversion and label lock → iteration 5 baseline/investigation/repair/retest → iteration 6 → iteration 7 → prior-batch regression checks → demo rehearsal → 156 passing unit tests → scanner freeze → final manifest freeze and acquisition verification → one final validation → this report. The ten MCP-DecayBench candidate servers remain unused for a possible future reserve, subject to label and overlap review. The consumed 33-artifact set must become development data if its results guide a future version.

Development baseline → repair → retest

Iteration	Fixtures	Detected before	Detected after	False alarms	Retest precision	Retest recall
5	78	2/48	45/48	2 → 0	100.0%	93.8%
6	24	8/12	12/12	2 → 0	100.0%	100.0%
7	20	2/10	9/10	0 → 0	100.0%	90.0%

These repaired scores use exposed development cases. They must not be presented as final-validation performance. Earlier batches showed no scoring regression.

Final validation protocol

The final evaluation used 33 pinned public artifacts from seven repositories, with 50 explicit artifact/rule labels (30 positive, 20 negative). The unit is a rule label, not a unique vulnerability incident. A file can have multiple labels. Only labeled rule outcomes enter precision and recall. All 33 artifacts were hash-verified before a single guarded scan. A consumed marker prevents reuse as an untouched final test. The optional LLM reviewer was disabled; no target program was executed. There were no scan errors. The code was frozen before final predictions and has not changed since.

Actual label	Detected	Not detected
Positive	13	17
Negative	1	19

Precision = TP / (TP + FP). Recall = TP / (TP + FN). False-positive rate = FP / (FP + TN). A missed positive reduces recall even when precision is high.

Category results and coverage

Category	Positive labels	Negative labels	TP	FN	FP	TN	Recall	Coverage
MCP01	5	4	3	2	0	4	60.0%	Scored
MCP02	1	0	0	1	0	0	0.0%	Scored
MCP03	5	5	3	2	0	5	60.0%	Scored
MCP04	0	0	0	0	0	0	—	Not covered
MCP05	10	7	4	6	1	6	40.0%	Scored
MCP06	4	0	1	3	0	0	25.0%	Scored
MCP07	1	1	0	1	0	1	0.0%	Scored
MCP08	0	0	0	0	0	0	—	Not covered
MCP09	2	0	0	2	0	0	0.0%	Scored
MCP10	0	0	0	0	0	0	—	Not covered
MCP11	0	0	0	0	0	0	—	Not covered
MCP12	0	0	0	0	0	0	—	Not covered
MCP13	2	3	2	0	0	3	100.0%	Scored
MCP14	0	0	0	0	0	0	—	Not covered
MCP15	0	0	0	0	0	0	—	Not covered
MCP16	0	0	0	0	0	0	—	Not covered

What the failures show

Six of the 17 missed labels occur in five files where no tool was recognized. This is evidence of registration/parser coverage gaps, although it is not a complete causal diagnosis. Other misses remain in scope detection, dynamic execution, contextual content, permissive binding, and confusable names. The one false positive is a command-injection label in the borderline JavaScript fixture. All final artifacts were scanned individually; project-wide and cross-file behaviors are not comprehensively evaluated by that protocol. The 100% synthetic retest contrasts with 43.3% final recall: success on constructed examples does not establish generalization. No final failures were used to repair this frozen version.

Limits of the evidence

The final labels cover eight MCP categories and ten specific rules. MCP04, MCP08, MCP10, MCP11, MCP12, MCP14, MCP15, and MCP16 lack final scored coverage. The 37 unscored artifact/rule predictions have unknown correctness; 92.9% precision applies only to the explicitly labeled scope. Small, correlated public fixtures do not establish population-wide performance. Source labels and assistant mappings need independent human review. No exact file-hash overlap was found against 779 distinct recorded development/primary hashes; this does not rule out semantic/template overlap.

▶ All final missed labels and false alarms

Show how it works

Open the five-step demo · Presenter notes · All 50 scored decisions · Portable evidence archive

Reproduce the demonstration with `python presentation_demo.py`. Do not rerun the final reserve during a presentation.

Reproducibility

156 unit tests passed. Scanner fingerprint:

`e4f1c74c15f3b8a9fb78764009ca33583daf9a3be7ba1107cadfd21d8f2f61bf`

Final manifest: `1f0dbec84b0a7c328ad518ddc03de8d293f3dc04bfa8e265c87b35d3d16b5651`

Scenario references: MCP security guidance; Python subprocess security.